



Avaliação Final – Testes de Software (Contexto Logística)



Duração total: 3 horas

- Parte 1: Teórica → 30 min
 - Parte 2: Prática → 2h30
-



PARTE 1 – MULTIPLA ESCOLHA (Responder no Link abaixo)

<https://forms.gle/u5LGoYWmMzYsUDuo7>



PARTE 2 – EXERCÍCIO PRÁTICO



Cenário

Você foi contratado por uma empresa de logística, inspirada em marketplaces como o **Mercado Livre**.

Sua missão é implementar e testar um sistema responsável por **processar envios de pacotes**.



Objetivo do exercício

Criar e testar um serviço responsável por:

- validar envio
- calcular frete
- simular envio para transportadora
- retornar status do envio

Regra de Negócio

Um envio deve seguir as regras:

◆ **1. Peso máximo permitido**

- máximo: 50 kg
- acima disso → erro

◆ **2. CEP obrigatório**

- não pode ser vazio

◆ **3. Frete**

- até 10 kg → R\$ 20
- acima de 10 kg → R\$ 40

◆ **4. Transportadora**

- deve ser chamada para processar o envio
- pode aprovar ou recusar



Código Base



EnvioService.ts

```
export class EnvioService {
  constructor(private transportadoraService: any) {}

  processarEnvio(peso: number, cep: string): string {
    if (!cep) {
      throw new Error('CEP obrigatório');
    }

    if (peso > 50) {
      throw new Error('Peso excede o limite permitido');
    }

    const frete = peso <= 10 ? 20 : 40;

    const aprovado = this.transportadoraService.enviar(peso, cep, frete);

    if (!aprovado) {
      throw new Error('Envio recusado pela transportadora');
    }

    return 'Envio processado com sucesso';
  }
}
```

TAREFAS

◆ 1. Criar testes unitários

Criar testes para:

✓ Caso 1 – Envio válido

- peso válido
- CEP válido
- transportadora aprova
- deve retornar sucesso

✗ Caso 2 – CEP vazio

- deve lançar erro
- transportadora NÃO deve ser chamada

✗ Caso 3 – Peso acima do limite

- deve lançar erro
- transportadora NÃO deve ser chamada

✗ Caso 4 – Envio recusado

- transportadora retorna false
- deve lançar erro

◆ 2. Usar mock da transportadora

Você deve criar:

```
const mockTransportadora = {  
  enviar: jest.fn()  
};
```

◆ 3. Validar chamadas

Deve usar:

```
toHaveBeenCalled()  
toHaveBeenCalledWith()  
not.toHaveBeenCalled()
```

◆ 4. Criar teste com spy

Testar com implementação real da transportadora:

```
jest.spyOn(...)
```

◆ 5. Aplicar boas práticas

- nomes claros
- padrão AAA
- testes independentes